

Guía paso a paso — Repositorio para una CNN 3D de triaje cerebral (tumor / sano)

Enfoque: encoder 3D **preentrenado** (FOMO260K + pesos de AMAES) + **fine-tuning** con datos etiquetados. Documento de trabajo.

0. Idea del enfoque (en 4 líneas)

1. **Preentrenar** = aprender "cómo es un cerebro en RM" a partir de muchísimos volúmenes sin etiqueta. No lo haces tú: **descargas un encoder 3D ya preentrenado** (AMAES, entrenado sobre FOMO260K: 260.927 scans de 910 fuentes).
2. **Fine-tuning** = enseñarle TU tarea (tumor / sano) con un conjunto **etiquetado**.
3. Para que aprenda **la lesión** y no "de qué dataset viene": que en los datos de fine-tuning **cada clase venga de varias fuentes mezcladas** (o ambas clases del mismo origen). Así "la fuente" deja de predecir la etiqueta y lo único consistente que separa las clases es el tumor.
4. Resultado: una CNN 3D que distingue sano/tumor de verdad, y con N pequeño etiquetado basta porque el encoder ya sabe de cerebros.

1. Crear el repositorio desde cero

Cada comando lleva, debajo, qué hace.

```
# 1) Crear la carpeta del proyecto y entrar
mkdir brain-mri-triage-3d && cd brain-mri-triage-3d

# 2) Inicializar git (control de versiones)
git init

# 3) Entorno de Python aislado (no mezclar librerías). Puedes reutilizar tu env 'igsan'.
python -m venv .venv && source .venv/bin/activate

# 4) Crear el esqueleto de carpetas de golpe
mkdir -p configs data/raw data/processed models/pretrained \
  src/data src/models src/training src/evaluation \
  scripts notebooks docs

# 5) Convertir src y subcarpetas en paquetes de Python
touch src/__init__.py src/data/__init__.py src/models/__init__.py \
  src/training/__init__.py src/evaluation/__init__.py
```

```
# 6) .gitignore (que NO suba datos, pesos ni resultados pesados)
printf "data/\nmodels/pretrained/\noutputs/\n.venv/\n__pycache__/\n*.pyc\
\n.ipynb_checkpoints/\n" > .gitignore

# 7) Librerías necesarias (pila 3D)
printf "torch\nmonai\nnibabel\nnumpy\npandas\nscikit-learn\nmatplotlib\nhuggingface_hub\
\nqdm\npyyaml\n" > requirements.txt

# 8) Instalarlas
pip install -r requirements.txt

# 9) README mínimo
printf "# brain-mri-triage-3d\nCNN 3D de triaje (tumor/sano) con encoder preentrenado
(FOM0260K/AMAES) + fine-tuning.\n" > README.md

# 10) Primer commit (foto inicial del repo)
git add . && git commit -m "Estructura inicial del repositorio"
```

2. Estructura de carpetas (definitiva)

Una frase por carpeta:

```
brain-mri-triage-3d/
├─ README.md           # Qué es el proyecto en 2 líneas
├─ requirements.txt     # Librerías necesarias
├─ .gitignore          # Qué NO se sube a git (datos, pesos, resultados)
├─ configs/
│   └─ finetune.yaml    # Todos los ajustes del fine-tuning en un sitio
├─ data/               # (no se sube a git)
│   └─ raw/             # Volúmenes descargados tal cual (NIfTI)
│   └─ processed/       # Volúmenes ya normalizados + metadata.csv
├─ models/
│   └─ pretrained/      # Pesos del encoder AMAES descargado (no se sube)
├─ src/
│   └─ data/
│       └─ prepare.py   # raw -> processed + metadata.csv (ruta, label, fuente)
│       └─ dataset.py   # Carga los volúmenes 3D y los da al modelo
│       └─ splits.py    # Reparte train/val/test (agrupado por sujeto, congelado)
│   └─ models/
│       └─ encoder.py   # Carga el encoder 3D preentrenado (AMAES)
│       └─ classifier3d.py# Encoder + cabeza de clasificación (1 logit: tumor/sano)
├─ training/
│   └─ finetune.py      # Fine-tuning del modelo y guardado del mejor checkpoint
├─ evaluation/
│   └─ evaluate.py      # Métricas: AUC, sensibilidad, especificidad, IC95%
├─ scripts/
│   └─ download_weights.sh# Descarga los pesos de AMAES
│   └─ run_all.sh       # Ejecuta el pipeline entero en orden
└─ notebooks/
```

```
| └─ eda.ipynb          # Exploración rápida de los datos
| └─ docs/              # Documentación y borradores
```

3. El proceso, explicado claro (preentrenar vs fine-tuning)

Fase	Qué hace	De dónde sale
Preentrenamiento	Aprende rasgos generales de cerebro en RM con MUCHÍSIMOS volúmenes sin etiqueta	No lo haces tú: descargas los pesos de AMAES (entrenados sobre FOMO260K)
Fine-tuning	Aprende TU tarea concreta: tumor / sano	Lo haces tú, con un set etiquetado
Evaluación	Mide si distingue de verdad	Conjunto de test separado

La condición clave: el preentrenamiento NO crea etiquetas; sigues necesitando un conjunto etiquetado para el fine-tuning. Lo que consigue es que ese conjunto, aunque sea pequeño, **baste**, porque el encoder ya sabe de cerebros.

4. Paso a paso: pesos de AMAES + datos FOMO260K

4.1 Descargar el encoder 3D preentrenado (AMAES)

```
# Clonar el repositorio de AMAES (código + checkpoints + documentación)
git clone https://github.com/asbjrnmunk/amaes.git external/amaes

# Seguir su README para descargar el checkpoint del encoder ResEnc preentrenado
# y colocarlo en models/pretrained/
```

Repos de referencia: github.com/asbjrnmunk/amaes (pesos + código) y github.com/fomo25/baseline-codebase (pipeline de preentrenar + fine-tuning ya montado, del reto FOMO25).

Verificar: formato exacto del checkpoint, framework (AMAES se apoya en estilo nnU-Net/nnssl) y cómo cargar solo el encoder. Hazlo siguiendo su README antes de programar nada.

4.2 Conseguir los datos etiquetados

Opción B (la preferente, si las etiquetas existen): FOMO260K.

- Está en Hugging Face: [FOMO-MRI/FOMO260K](#). Formato NIfTI (.nii.gz). Licencia CC-BY-NC-SA-4.0 (uso académico OK).

- Trae metadatos: `participants.tsv` (diagnóstico: control / tumor...), `mri_info.tsv`, `mapping.tsv`.
- Como mezcla 910 fuentes y ambas clases salen de fuentes solapadas, "la fuente" no predice la etiqueta → entrenar ahí fuerza a aprender la lesión.

```
# Login en Hugging Face (una sola vez)
huggingface-cli login

# NO descargues 1,87 TB enteros. Baja solo lo que necesites (subconjunto).
# Ejemplo: descargar los metadatos primero para ver las etiquetas
huggingface-cli download FOMO-MRI/FOMO260K --repo-type dataset \
  --include "*participants.tsv" --local-dir data/raw/fomo_meta
```

Verificar antes de seguir: abre los `participants.tsv` y comprueba que de verdad distinguen **tumor vs control** de forma usable. Si es así, descargas solo los sujetos de esas dos clases.

Opción A (alternativa segura): un set etiquetado pequeño de fuente única (o con varias fuentes por clase). Sirve para el fine-tuning aunque sea de decenas de sujetos, porque el encoder ya está preentrenado.

4.3 Preparar los datos (`src/data/prepare.py`)

- Leer `participants.tsv` y construir `data/processed/metadata.csv` con columnas: `ruta`, `label` (1=tumor, 0=sano), `fuentes`, `subject_id`.
- Homogeneizar cada volumen: reorientar a RAS, remuestrear a 1 mm³, recortar/rellenar a una forma fija, normalizar (z-score sobre tejido). (El encoder de AMAES espera un preprocesado concreto: ajústate al que indique su repo.)

4.4 Repartir en train/val/test (`src/data/splits.py`)

- Estratificado por clase, **agrupado por `subject_id`** (que ningún sujeto caiga en dos particiones), semilla fija (42), y **congelar** el split en un JSON.

4.5 Montar el modelo (`src/models/`)

- `encoder.py`: carga el encoder 3D preentrenado de AMAES desde `models/pretrained/`.
- `classifier3d.py`: encoder + una **cabeza de clasificación** (pooling global → capa lineal → 1 logit). Salida = `sigmoid(logit)` = probabilidad de tumor.

4.6 Fine-tuning (`src/training/finetune.py`)

- Estrategia recomendada: empezar con el **encoder congelado** (entrenar solo la cabeza)

y luego, si va bien, descongelar con learning rate bajo.

- Pérdida `BCEWithLogitsLoss`; optimizador AdamW; guardar el mejor checkpoint por **balanced accuracy** con sensibilidad mínima; semilla fija; split congelado.

4.7 Evaluar (`src/evaluation/evaluate.py`)

- AUC, PR-AUC, sensibilidad, especificidad, F1, matriz de confusión, e **intervalos de confianza por bootstrap**.
- El umbral de decisión se elige en validación y se aplica a test.

5. Orden de ejecución (el flujo)

```
bash scripts/download_weights.sh          # 1) pesos de AMAES -> models/pretrained/
python -m src.data.prepare                 # 2) raw -> processed + metadata.csv
python -m src.data.splits                  # 3) train/val/test (congelado)
python -m src.training.finetune --config configs/finetune.yaml # 4) fine-tuning
python -m src.evaluation.evaluate --checkpoint outputs/best.pt # 5) métricas
```

6. Notas prácticas (para no estrellarte)

- **No preentrenes tú:** descarga los pesos. Preentrenar 260k volúmenes es inviable en tu equipo; el fine-tuning sí es asumible (GPU AMD ROCm).
- **No bajes 1,87 TB:** descarga metadatos primero y luego solo el subconjunto de sujetos tumor/control que vayas a usar.
- **Etiquetas:** el preentrenamiento no las da; necesitas un set etiquetado para el fine-tuning. Verifica las de FOMO260K antes de comprometerte (Opción B); si no, usa la Opción A.
- **Licencia:** FOMO260K es CC-BY-NC-SA-4.0 (no comercial) → válido para un TFG; cita la fuente.
- **Curva de aprendizaje:** foundation models 3D + fine-tuning es más avanzado; apóyate en el README de AMAES y en la baseline de FOMO25.

7. Enlaces

- AMAES (pesos + código): github.com/asbjrnmunk/amaes
- FOMO25 baseline (pipeline preentrenar+fine-tuning): github.com/fomo25/baseline-codebase
- Reto FOMO25: fomo25.github.io

- Dataset FOMO260K: FOMO-MRI/FOMO260K (Hugging Face)
- Paper del dataset: arxiv.org/abs/2506.14432

Documento generado como guía de trabajo. Los pasos de descarga de pesos y de etiquetas de FOMO260K deben verificarse contra los repos/HF oficiales antes de programar.